

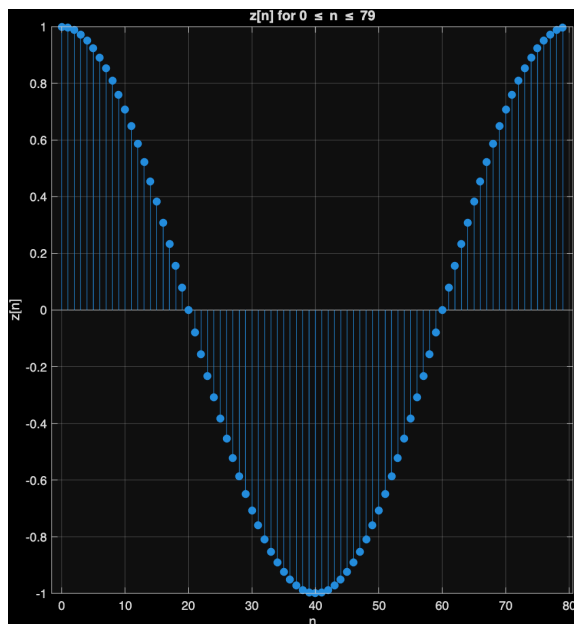
lab3 results document:

Name: Anas Khmais Hamrouni

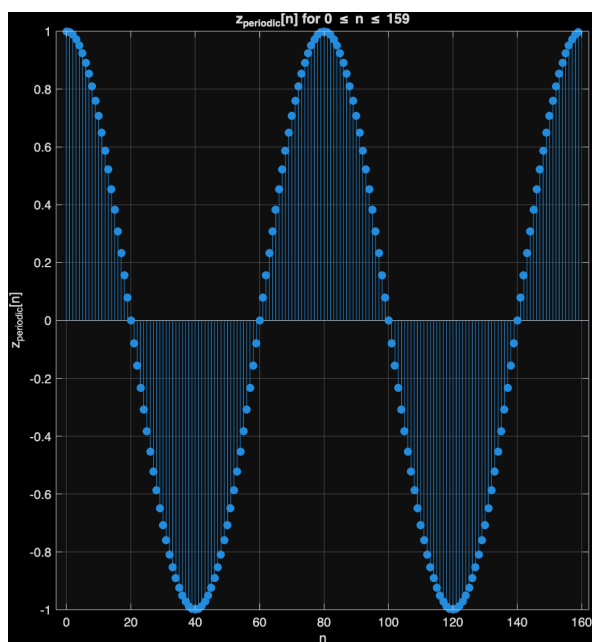
Names of all students you discussed this lab with: none

Section 1:

Question 1: Include your stem plot of $z[n]$ vs n for $0 \leq n \leq 79$, below:

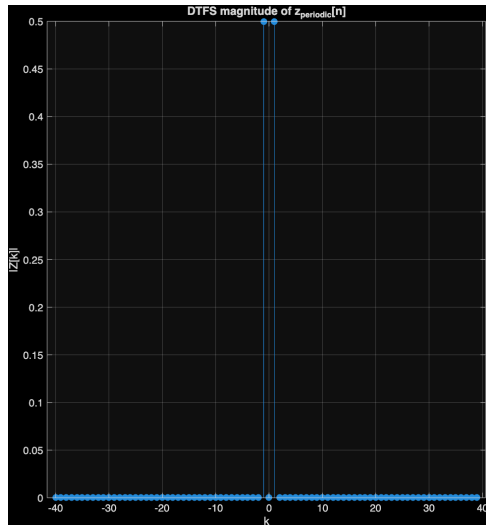


Question 2: Plot $z_{\text{periodic}}[n]$ vs n for $0 \leq n \leq 159$. Find a simple formula for $z_{\text{periodic}}[n]$ as a function of n .

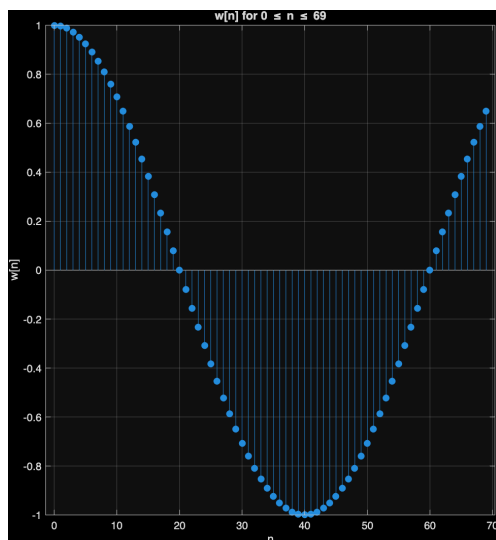


$$z_{\text{periodic}}[n] = \cos(4\pi n/160), \quad n=0,1,\dots,159$$

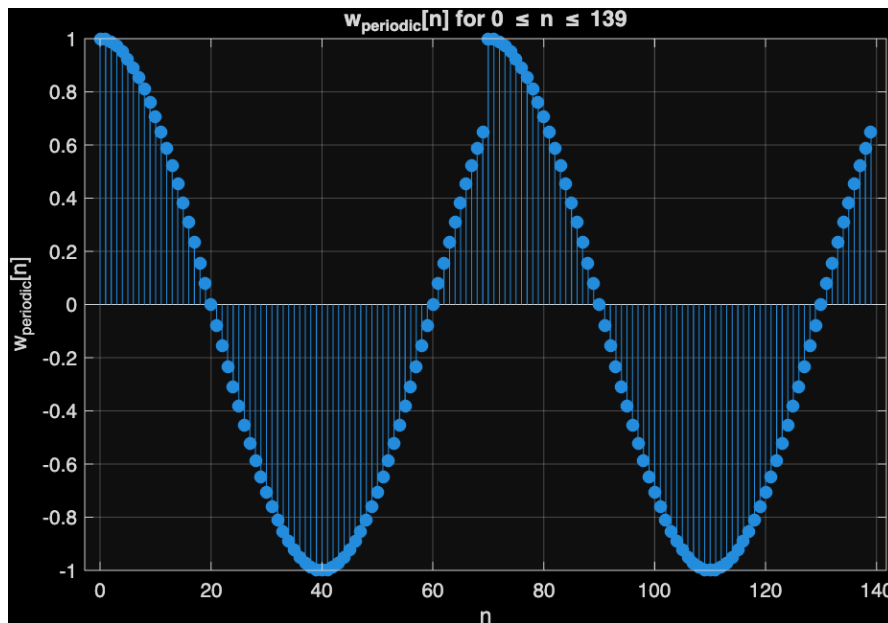
Question 3: Plot the magnitude of the discrete-time Fourier series coefficients of z_{periodic} . Include one period of the coefficients in your plot, and center the plot at $k = 0$.



Question 4: Make a stem plot of $w[n]$ vs n for $0 \leq n \leq 69$.



Question 5: Plot $w_{\text{periodic}}[n]$ vs n for $0 \leq n \leq 139$. Comment on the difference between the plots of w_{periodic} and z_{periodic}

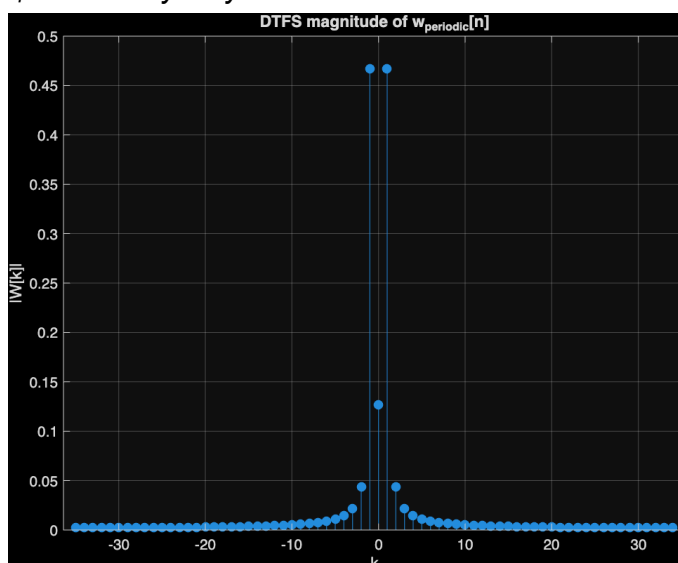


z_{periodic} repeats a block of length 80 samples, so its pattern over 0–159 consists of exactly two identical 80-sample segments.

w_{periodic} repeats a block of length 70 samples, so over 0–139 we see two repeats of a 70-sample cosine segment with 10-sample gaps at the end of each 70-sample period (since z is defined over 80 samples and w only over 70).

z_{periodic} looks like a longer continuous cosine window repeated with period 80, while w_{periodic} looks like a shorter cosine window repeated more often (period 70), so its repetitions do not align with those of z_{periodic} .

Question 6: Plot the magnitude of the discrete-time Fourier series coefficients of w_{periodic} . Include one period of the coefficients in your plot, and center the plot at $k = 0$. How does the plot differ from the plot of the magnitude of the discrete-time Fourier series coefficients of z_{periodic} ? Why do you think this is the case?

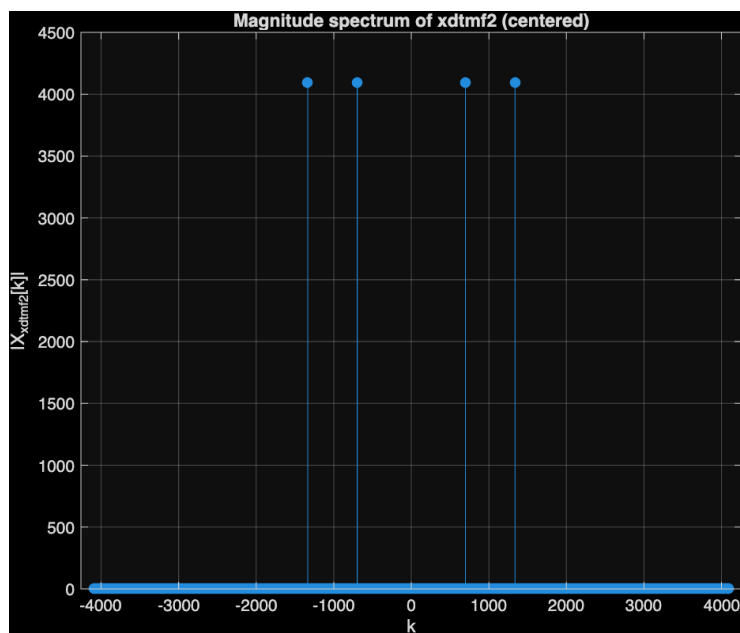
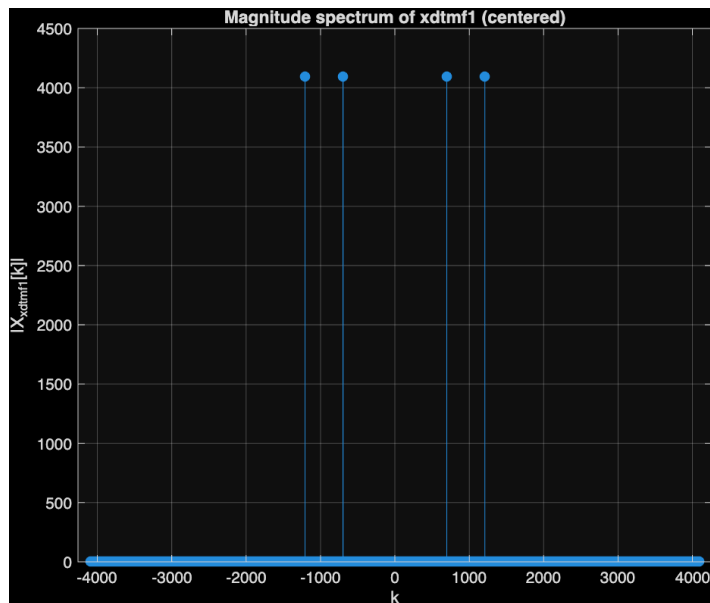


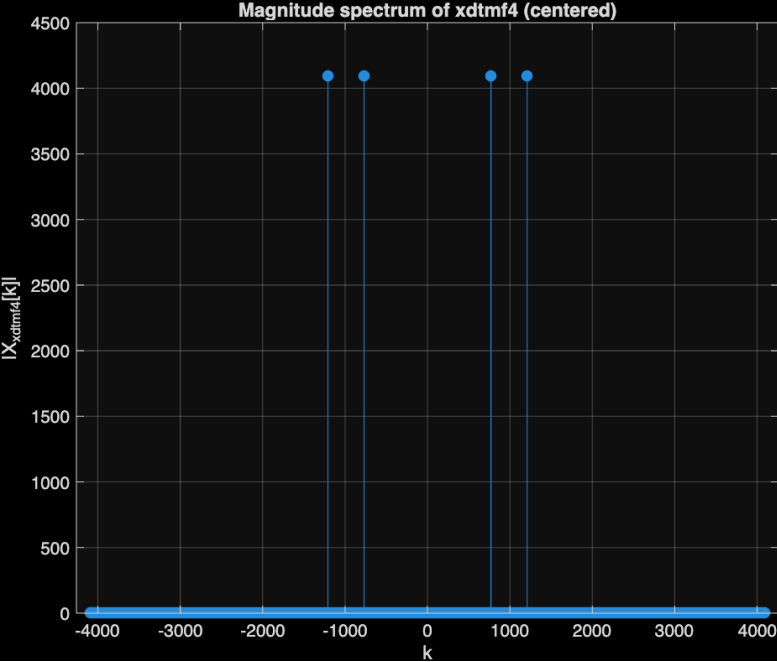
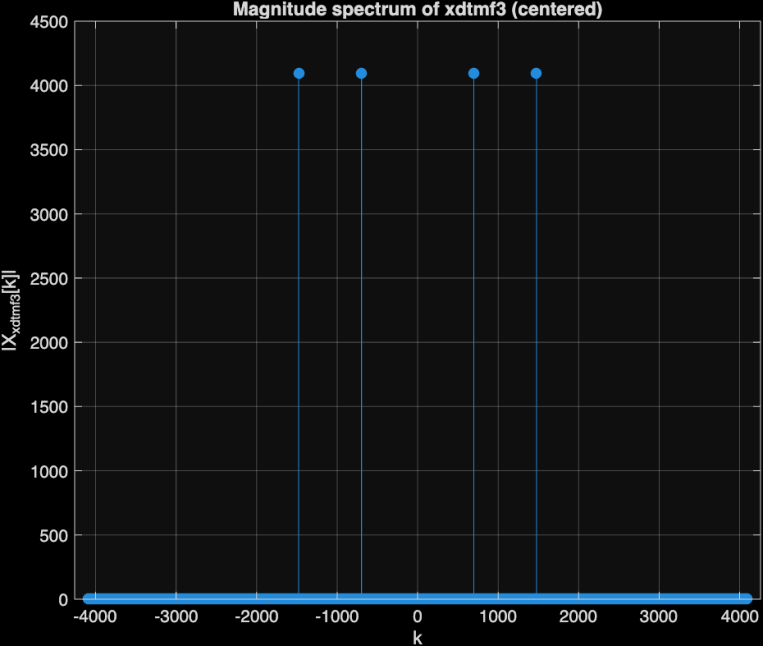
The magnitude spectrum of z_{Periodic} has non-zero coefficients at indices corresponding to the cosine frequency sampled over 80 points, while the spectrum of w_{Periodic} is computed over 70 points and has its non-zero coefficients at different index locations and with different magnitudes.

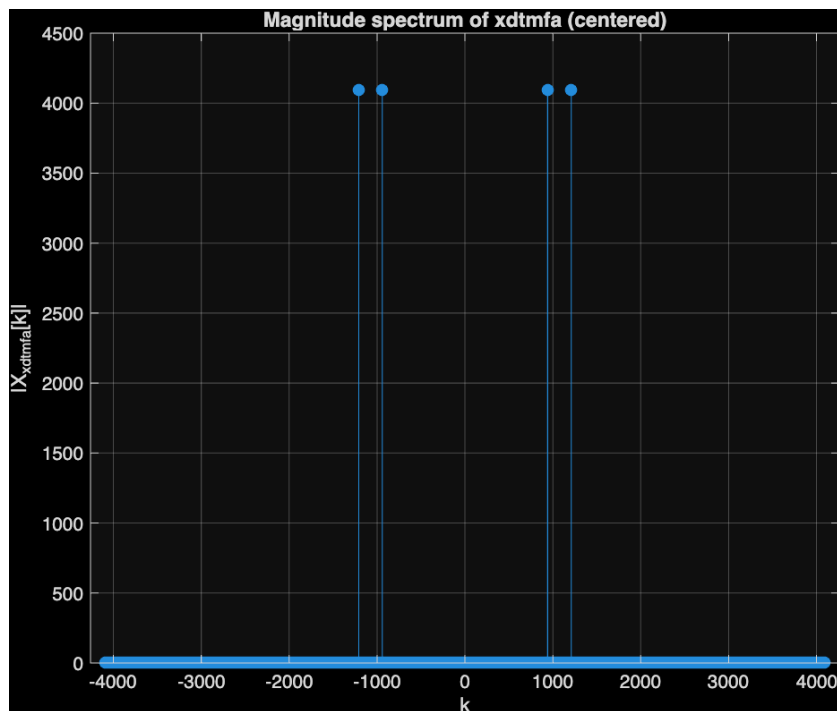
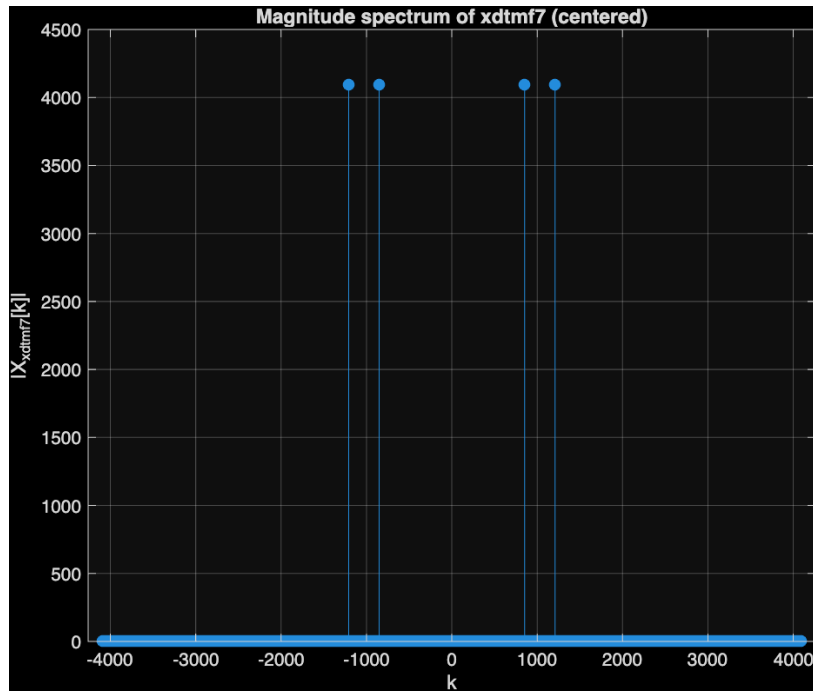
Because $w[n]$ is a shorter window of the same underlying cosine, its time support and period differ, which changes the spacing and height of frequency components in the DTFS. We observe different frequency resolution and different window length, leading to different spectral shapes.

Section 2:

Question 2(b): Plot the magnitude of the discrete-time Fourier series coefficients of the signal. Use a stem plot. Arrange the horizontal axis so that the zero frequency component is shown in the center of the plot.







Question 3: Determine the frequencies ω_{ri} for $i = 1, 2, 3, 4$ and ω_{cj} for $j = 1, 2, 3$.

From the spectra of the clean DTMF signals, the distinct tone frequencies present are approximately 697, 770, 852, 941, 1209, 1336, and 1477 Hz.

The four lower values (697, 770, 852, 941 Hz) correspond to the row frequencies $f_{r1}, f_{r2}, f_{r3}, f_{r4}$, and the three higher values (1209, 1336, 1477 Hz) correspond to the column frequencies f_{c1}, f_{c2}, f_{c3} .

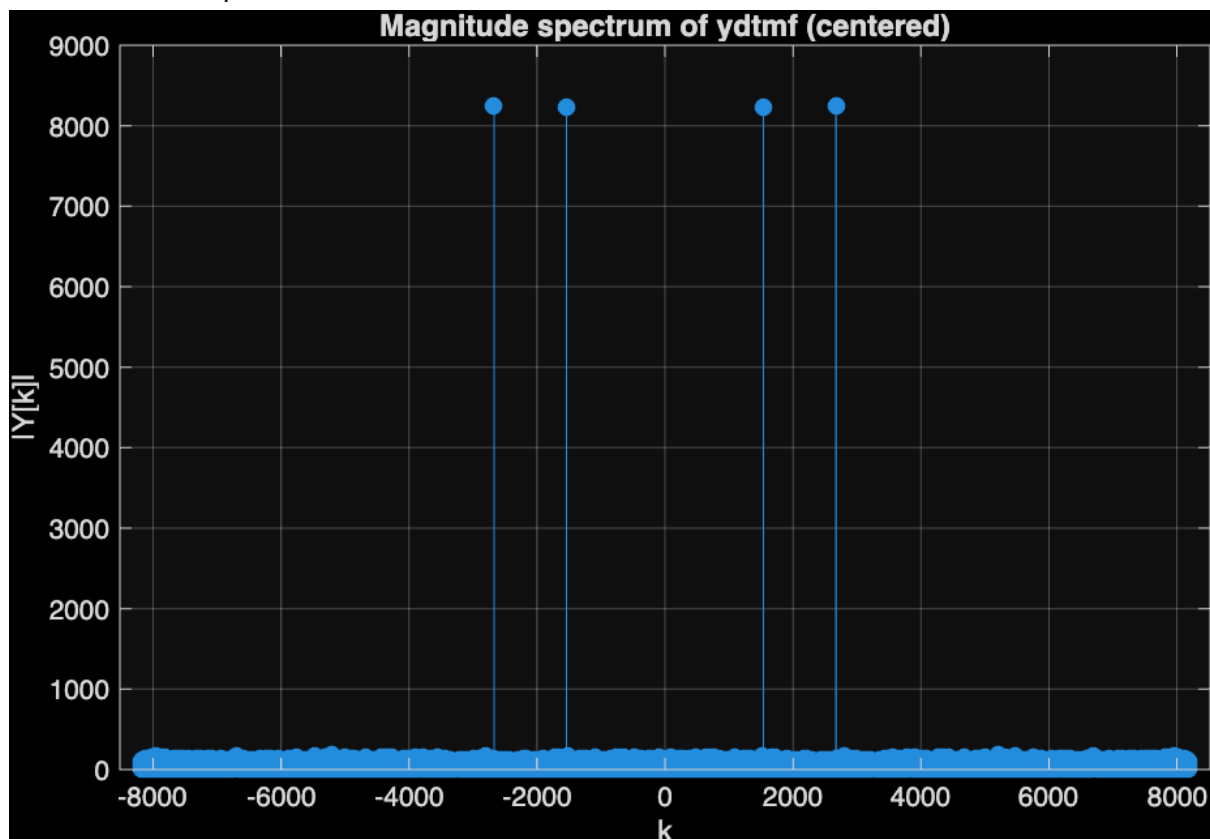
Converting to discrete-time radian frequencies using $\omega = 2\pi f / F_s$ with $F_s = 8192$ Hz

gives approximately $\omega_r \approx [0.5346, 0.5906, 0.6535, 0.7217]$ rad and $\omega_c \approx [0.9273, 1.0247, 1.1328]$ rad.

Question 4: What is the duration of the signal `ydtmf`?

`ydtmf` length = 16384 samples, duration ≈ 2.0000 s

Question 6: Plot the magnitude of the discrete-time Fourier series coefficients of `ydtmf`. Use a stem plot. Arrange the horizontal axis so that the zero frequency component is shown in the center of the plot.



Question 7: Find the indices of the discrete-time Fourier series coefficient of `ydtmf` that have large magnitude.

The noisy DTMF signal `ydtmf` has large-magnitude DTFS coefficients corresponding to a low-frequency component near 770 Hz and a high-frequency component near 1336 Hz (after folding conjugate peaks into the baseband). These match the row–column pair (row 2, column 2) in the DTMF table, so `ydtmf` corresponds to the key “2”.

Question 8: Which button press does `ydtmf` correspond to?

From the spectrum of the noisy DTMF signal `ydtmf`, the two dominant frequency components correspond to approximately 770 Hz (row 2) and 1336 Hz (column 2) after accounting for mirrored peaks. These frequencies match the standard DTMF row–column pair for the key “2”, so `ydtmf` corresponds to button 2 on the keypad.

Section 3:

Question 2: At the sampling rate F_{st} , how many samples correspond to 30ms of audio?

$M = 2880$ samples (30 ms), $P = 402$ segments, $\text{length}(y) = 1157760$

Question 6: Explain how the command

```
Ysegtrunc = Yseg.*(abs(Yseg) > threshold*maxval);
```

works.

The expression `abs(Yseg) > threshold*maxval` constructs a logical mask that is 1 wherever the magnitude of a coefficient exceeds the threshold and 0 elsewhere, multiplying this mask elementwise with `Yseg` keeps only the large coefficients and sets all others to zero.

Question 7: By typing `whos Ysegtrunc` find out how much memory this variable requires.

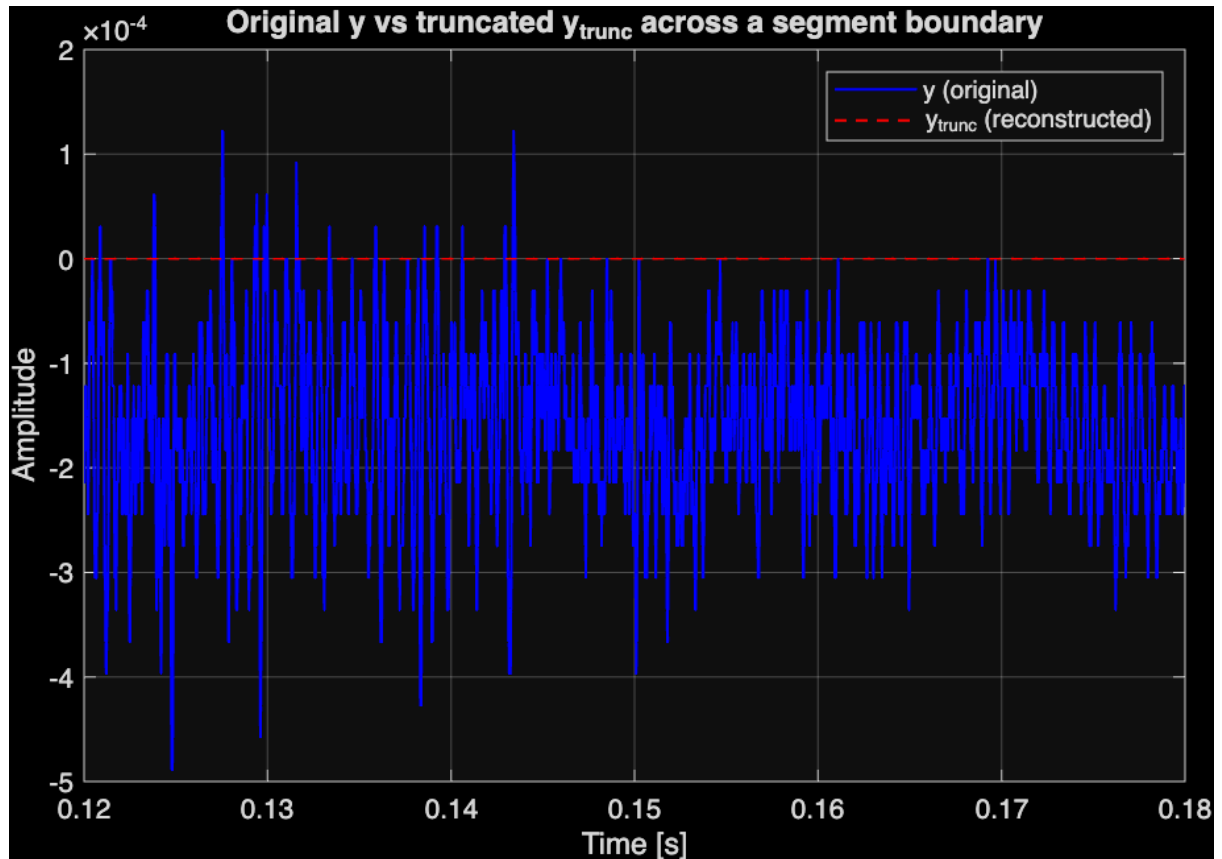
Compare this with the memory required to store `y`.

After converting the truncated coefficient matrix to sparse format, it uses about 814.3 kB compared to 9045.0 kB for the original signal `y`, demonstrating a significant reduction in storage.

Question 8: Choose a time- interval on which `ytrunc` and `y` are both non-zero, that crosses a 30ms segment boundary. Over this time-interval, plot both signals. What do you notice? Plot the difference between `ytrunc` and `y`. What do you notice? What do you think is the basic problem with our compression method?

8c) When listening to the reconstructed signal `ytrunc`, the audio sounds noticeably noisier and less clean than the original trumpet signal `y`, indicating that the coefficient truncation and blockwise approximation introduce audible artefacts.

8d)



In the selected interval crossing a 30 ms boundary, the original signal y shows small-amplitude waveform variations, whereas the reconstructed signal y_{trunc} is almost completely flattened around zero and does not follow the original shape. This indicates that the chosen threshold removed nearly all Fourier coefficients for these segments, so the blockwise approximation fails to capture even low-level details and introduces strong distortion. This illustrates a limitation of the simple blockwise Fourier compression: when too many coefficients are discarded, some segments are represented by almost no information, leading to severe loss of signal structure despite the large memory savings.

Code for section 1:**Code from section 1.2:**

Paste your script in here.

```
Fs = 160;
Ts = 1/Fs;
n = (0:79).';

% sampled cosine x_s[n] = cos(4π n / 160)
xs = cos(4*pi*(n*Ts));

% simple unit step implementation
u = @(n) double(n >= 0);

% finite-length window: keep samples 0..79, zero elsewhere
z = xs .* (u(n) - u(n-80));

figure;
stem(n, z, 'filled');
xlabel('n');
ylabel('z[n]');
title('z[n] for 0 \leq n \leq 79');
grid on;

% ---- periodic extension of z[n] with period 80 ----
N_z = 80;
n2 = (0:159).';

% map indices 0..159 into one period 0..79
idx_mod = mod(n2, N_z) + 1;
z_periodic = z(idx_mod);

figure;
stem(n2, z_periodic, 'filled');
xlabel('n');
ylabel('z_{periodic}[n]');
title('z_{periodic}[n] for 0 \leq n \leq 159');
grid on;

% ---- DTFS of z_periodic via FFT over one period ----
Xz = (1/N_z) * fft(z_periodic(1:N_z));

% frequency indices centered at zero
k = -floor(N_z/2):(N_z-1-floor(N_z/2));

figure;
```

```

stem(k, fftshift(abs(Xz)), 'filled');
xlabel('k');
ylabel('|Z[k]|');
title('DTFS magnitude of  $z_{\text{periodic}}[n]$ ');
grid on;

% ---- construct w[n] on a shorter window 0..69 ----
n_w = (0:69).';
xs_w = cos(4*pi*(n_w*Ts));

% same cosine, different window length (70 samples)
w = xs_w .* (u(n_w) - u(n_w-70));

figure;
stem(n_w, w, 'filled');
xlabel('n');
ylabel('w[n]');
title('w[n] for  $0 \leq n \leq 69$ ');
grid on;

% ---- periodic extension of w[n] with period 70 ----
N_w = 70;
n5 = (0:139).';

% map indices 0..139 into one period 0..69
idx_mod_w = mod(n5, N_w) + 1;
w_periodic = w(idx_mod_w);

figure;
stem(n5, w_periodic, 'filled');
xlabel('n');
ylabel('w_{periodic}[n]');
title('w_{periodic}[n] for  $0 \leq n \leq 139$ ');
grid on;

% ---- DTFS of w_periodic via FFT over one period ----
Xw = (1/N_w) * fft(w_periodic(1:N_w));
k_w = -floor(N_w/2):(N_w-1-floor(N_w/2));

figure;
stem(k_w, fftshift(abs(Xw)), 'filled');
xlabel('k');
ylabel('|W[k]|');
title('DTFS magnitude of w_{periodic}[n]');
grid on;

```

Code for section 2

Code from section 2.1:

Paste your script in here.

```
load('dtmfclean.mat');

Fs = 8192;
N = length(xdtmf1);
Ts = 1/Fs;
n = (0:N-1).';
t = n*Ts;

% cell array for looping over all clean DTMF signals
signals = {xdtmf1, xdtmf2, xdtmf3, xdtmf4, xdtmf7, xdtmf8};
names = {'xdtmf1', 'xdtmf2', 'xdtmf3', 'xdtmf4', 'xdtmf7', 'xdtmf8'};

% relative threshold for "large" FFT coefficients
large_factor = 0.3;

for i = 1:numel(signals)
    x = signals{i};
    sname = names{i};

    % listen to current clean DTMF tone
    fprintf('Playing %s ...\n', sname);
    soundsc(x, Fs);
    pause(1.2);

    % DTFS via FFT and magnitude spectrum
    X = fft(x);
    magX = abs(X);

    % frequency indices centered at zero
    k = -floor(N/2):(N-1-floor(N/2));

    figure;
    stem(k, fftshift(magX), 'filled');
    xlabel('k');
    ylabel(['|X_{', sname, '}[k]|']);
    title(['Magnitude spectrum of ', sname, ' (centered)']);
    grid on;

    % find indices whose magnitude is above chosen fraction of max
    maxval = max(magX);
    idx_large = find(magX > large_factor*maxval);

    fprintf('%s: large-magnitude coefficient indices\n(0-based):\n', sname);
```

```

disp(idx_large.' - 1);

% store 0-based indices for later frequency calculation
large_idx_struct.(sname) = idx_large - 1;
end

% ---- Task 3: extract row/column frequencies from dominant bins
----

% manually collected dominant indices (0-based) for each clean
signal
k_xdtmf1 = [697 1209 6983 7495];
k_xdtmf2 = [697 1336 6856 7495];
k_xdtmf3 = [697 1477 6715 7495];
k_xdtmf4 = [770 1209 6983 7422];
k_xdtmf7 = [852 1209 6983 7340];
k_xdtmfa = [941 1209 6983 7251];

all_k = [k_xdtmf1;
         k_xdtmf2;
         k_xdtmf3;
         k_xdtmf4;
         k_xdtmf7;
         k_xdtmfa];

% keep only baseband copies (0..N/2) - actual physical tones
base_k = all_k(all_k <= N/2);

% distinct tone frequencies in Hz ( $F_s = N = 8192 \Rightarrow f = k$ )
f_candidates = unique(base_k);

disp('All distinct tone frequencies in Hz (from one period):');
disp(f_candidates.);

% split into row and column sets according to their values
row_freqs_Hz = [697 770 852 941];
col_freqs_Hz = [1209 1336 1477];

% corresponding discrete-time radian frequencies  $\omega = 2\pi f/F_s$ 
row_omegas_rad = 2*pi*row_freqs_Hz/Fs;
col_omegas_rad = 2*pi*col_freqs_Hz/Fs;

fprintf('Row frequencies (Hz): '); disp(row_freqs_Hz);
fprintf('Column frequencies (Hz): '); disp(col_freqs_Hz);
fprintf('Row  $\omega_{ri}$  (radians):\n');
disp(row_omegas_rad.);
fprintf('Column  $\omega_{cj}$  (radians):\n');

```

```

disp(col_omegas_rad. ');

% ---- Task 4-7: analyse noisy DTMF signal ----

load('dtmfnoisy.mat');

N_y = length(ydtmf);
duration_y = N_y / Fs; % duration in seconds
fprintf('ydtmf length = %d samples, duration ≈ %.4f s\n', N_y,
duration_y);

% listen to noisy tone
soundsc(ydtmf, Fs);

% FFT and magnitude spectrum of noisy signal
Xy = fft(ydtmf);
magXy = abs(Xy);

N_y = length(ydtmf);
k_y = -floor(N_y/2):(N_y-1-floor(N_y/2));

figure;
stem(k_y, fftshift(magXy), 'filled');
xlabel('k');
ylabel('|Y[k]|');
title('Magnitude spectrum of ydtmf (centered)');
grid on;

% detect large-magnitude bins in noisy spectrum
maxval_y = max(magXy);
large_facY = 0.3;
idx_large_y = find(magXy > large_facY*maxval_y);

fprintf('ydtmf large-magnitude coefficient indices (0-based):\n');
disp(idx_large_y.' - 1);

% print approximate frequencies for those bins
for k0 = (idx_large_y.' - 1)
    f = k0*Fs/N_y;
    fprintf('k = %d -> f ≈ %.1f Hz\n', k0, f);
end

```

Code for section 3

Code from section 3.1:

Paste your script in here.

```
% read trumpet signal (use first channel if stereo)
[y, Fst] = audioread('trumpet.wav');
y = y(:,1);
fprintf('length(y) = %d samples, Fst = %.1f Hz\n', length(y),
Fst);

% choose segment length: 30 ms in samples and trim to full
segments
M = round(0.030 * Fst);
Ny = length(y);
P = floor(Ny / M);
y = y(1:M*P);
Ny = length(y);
fprintf('M = %d samples (30 ms), P = %d segments, length(y) =
%d\n', M, P, Ny);

% arrange signal into columns, each column is one 30 ms block
yseg = reshape(y, M, P);
fprintf('yseg is %d-by-%d (M-by-P)\n', size(yseg,1),
size(yseg,2));

% Fourier coefficients of each block (DTFS per segment)
Yseg = fft(yseg, [], 1);
fprintf('Yseg has same size as yseg: %d-by-%d\n', size(Yseg,1),
size(Yseg,2));

% global maximum magnitude across all coefficients
maxval = max(abs(Yseg), [], 'all');
fprintf('maxval = %.4e\n', maxval);

% keep only coefficients above fixed fraction of maxval
threshold = 0.01;
Ysegtrunc = Yseg .* (abs(Yseg) > threshold*maxval);
fprintf('applied threshold = %.4f * maxval\n', threshold);

% store truncated coefficients in sparse form to save memory
Ysegtrunc_sparse = sparse(Ysegtrunc);
info_y = whos('y');
info_Ysegtrunc_sparse = whos('Ysegtrunc_sparse');
fprintf('Task 7:\n');
fprintf('  y uses %.1f kB of memory\n',
info_y.bytes/1024);
fprintf('  Ysegtrunc_sparse uses %.1f kB of memory\n',
info_Ysegtrunc_sparse.bytes/1024);
```



```

% reconstruct segments from truncated coefficients
Ysegtrunc_full = full(Ysegtrunc_sparse);
yseg_trunc = ifft(Ysegtrunc_full, [], 1);

% stitch blocks back into a single time-domain signal
ytrunc = reshape(yseg_trunc, M*P, 1);

% listen to original vs compressed / reconstructed signal
soundsc(y, Fst);
soundsc(ytrunc, Fst);

%% compare waveforms across a segment boundary (Task 8d)

p = 5; % choose a segment index to inspect
n_start = (p-1)*M + 1;
n_end = (p+1)*M; % cover two neighbouring segments
n_range = n_start:n_end;

t = (n_range-1)/Fst;

figure;
plot(t, y(n_range), 'b', 'LineWidth', 1.0); hold on;
plot(t, real(ytrunc(n_range)), 'r--', 'LineWidth', 1.0);
xlabel('Time [s]');
ylabel('Amplitude');
title('Original y vs truncated y_{trunc} across a segment boundary');
legend('y (original)', 'y_{trunc} (reconstructed)');
grid on;

```